

tuto SDL : interface graphique en C.

Comment faire une fenêtre avec SDL :

```
//initialisation
if (SDL_Init(SDL_INIT_VIDEO) < 0)
{
    //gestion de l'erreur avec SDL
    SDL_LogError(stderr, "Erreur SDL_Init: %s", SDL_GetError());
    return EXIT_FAILURE;
}

//creation d'un renderer et d'une fenetre
SDL_Window *window = NULL;
SDL_Renderer *renderer = NULL;
if(0 != SDL_CreateWindowAndRenderer(500, 500, SDL_WINDOW_SHOWN,
    window, renderer))
{
    fprintf(stderr, "Erreur SDL_CreateWindowAndRenderer: %s",
        SDL_GetError());
    return -1;
}
```

Comment afficher une image avec SDL sur notre fenêtre :

```
//recuperer l'image
//fonction ecrite dans le fichier file.c
SDL_Texture *textureMenu = loadImage("goomba1.png", renderer);

//On recupere toute l'image en utilisant un rectangle
SDL_Rect src1 = {0, 0, 0, 0};

//On place l'image en haut a gauche
//l'image est de taille 40x40
SDL_Rect dst1 = { 0, 0, 40, 40 };

//on nettoie la fenetre
SDL_RenderClear(renderer);

//on positionne l'image sur la fenetre
//(position donne avec les rectangle SDL_Rect)
SDL_RenderCopy(renderer, textureMenu, &src1, &dst1);

//mise a jour de la fenetre
SDL_RenderPresent(renderer);
```

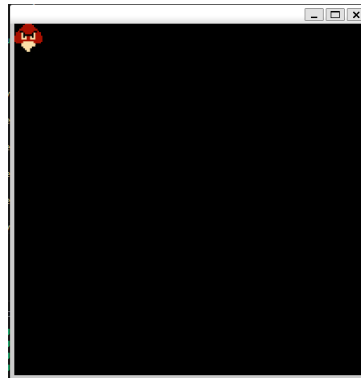


Figure 1: Rendu Goomba entier

Le rectangle dst (pour destination) permet de positionner l'image dans notre fenêtre, ici en prenant un rectangle plus grand, l'image est agrandie.

```
//recuperer l'image
//fonction ecrite dans le fichier file.c
SDL_Texture *textureMenu = loadImage("goomba1.png", renderer);

//On recupere toute l'image en utilisant un rectangle
SDL_Rect src1 = {0, 0, 0, 0};

//On place l'image en haut a gauche
//l'image est de taille 40x40
SDL_Rect dst1 = { 0, 0, 400, 400 };

//on nettoie la fenetre
SDL_RenderClear(renderer);

//on positionne l'image sur la fenetre
//(position donne avec les rectangle SDL_Rect)
SDL_RenderCopy(renderer, textureMenu, &src1, &dst1);

//mise a jour de la fenetre
SDL_RenderPresent(renderer);
```

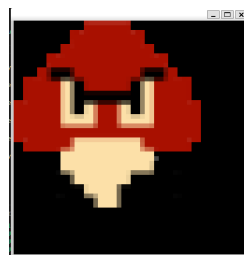


Figure 2: Rendu Goomba agrandi

Le rectangle src (pour source) permet de choisir un morceau de l'image de départ, ici on choisit le bout de l'image en bas à droite :

```
//recuperer l'image
//fonction ecrite dans le fichier file.c
SDL_Texture *textureMenu = loadImage("goomba1.png", renderer);
```

```
//On recupere toute l'image en utilisant un rectangle
SDL_Rect src1 = {20, 20, 20, 20};

//On place l'image en haut a gauche
//l'image est de taille 40x40
SDL_Rect dst1 = { 0, 0, 40, 40 };

//on nettoie la fenetre
SDL_RenderClear(renderer);

//on positionne l'image sur la fenetre
//(position donne avec les rectangle SDL_Rect)
SDL_RenderCopy(renderer, textureMenu, &src1, &dst1);

//mise a jour de la fenetre
SDL_RenderPresent(renderer);
```

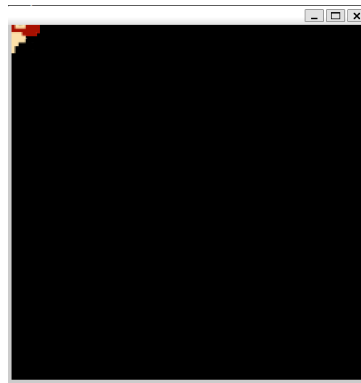


Figure 3: Rendu un bout de Goomba

Pour changer la fenêtre, il faut toujours utiliser les fonctions suivantes :

```
//on nettoie la fenetre
SDL_RenderClear(renderer);

//ici on decide de quoi mettre dans la fenetre

//mise a jour de la fenetre
SDL_RenderPresent(renderer);
```

Pour faire votre jeu, vous devez faire une boucle de jeu. En effet, chaque tour de boucle va générer une image du jeu (1 fps), ce qui va permettre de créer le rendu du jeu. La boucle de jeu (ou gameloop) est consituee :

- Input : évènement clavier, souris,
- Update game : on met à jour le jeu en fonction des inputs faite à l'étape précédente. Par exemple vous avez appuyé sur la flèche vers la droite, le personnage doit se déplacer vers la droite
- Render : On dessine toute la scène en 2d de notre jeu.

La gameloop est souvent une boucle quasi infinie (tant que le jeu n'est pas terminé).

```
SDL_Event events;
int continuer = 1;
while(continuer){
    //on recupere un evenement
    while (SDL_PollEvent(&events))
    {
        switch (events.type)
        {
            case SDL_QUIT:
                //on a clique sur la croix de la fenetre
                continuer = 0;
                //continuer passe a 0, la boucle se termine
                break;
            case SDL_KEYDOWN: //on a appuyé sur une touche
                switch(events.key.keysym.sym) //liste des touches appuyées ici
                {
                    case SDLK_1:
                        //on a appuyé sur la touche 1
                    }
                break;
            case SDL_KEYUP:
                switch(events.key.keysym.sym)
                {
                    case SDLK_1:
                        //on a relâché la touche 1
                    }
                break;
        }
    }
}
```

Dans le code précédent, vous voyez un exemple de gestion d'événements. La fonction `SDL_PollEvent` permet de récupérer l'événement en cours et de le stocker dans la variable `events`. On voit ensuite une suite de `switch case`, avec chaque case correspondant à un événement précis (une touche appuyée, la souris qui bouge, ...). Vous pouvez trouver la liste des événements sur le lien suivant : https://wiki.libsdl.org/SDL2/SDL_ScancodeAndKeycode (il faut prendre les keycode ici)